$count (aggregation stage)

Definition

`$count` Passes a document to the next stage that contains a count of the number of documents input to the stage.

This page describes the `$count` aggregation pipeline stage. For the `$count` aggregation accumulator, see `$count (aggregation accumulator)`.

Compatibility

`$count` You can use `$count` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

`$count` has the following syntax:

```
{ $count: <string> }
```

`<string>` is the name of the output field which has the count as its value. `<string>` must be a non-empty string, must not start with `$` and must not contain the `.` character.

Behavior

The return type is represented by the smallest type that can store the final value of count: `integer` → `long` → `double`

The `$count` stage is equivalent to the following `$group` and `$project` sequence:

```
db.collection.aggregate( [
   { $group: { _id: null, myCount: { $sum: 1 } } },
   { $project: { _id: 0 } }
] )
```

`myCount` is the output field that stores the count. You can specify another name for the output field.

If the input dataset is empty, `$count` doesn't return a result.

`db.collection.countDocuments()` wraps the `$group` aggregation stage with a `$sum` expression.

Examples

Create a collection named `scores` with these documents:

```
db.scores.insertMany( [
   { "_id" : 1, "subject" : "History", "score" : 88 },
   { "_id" : 2, "subject" : "History", "score" : 92 },
   { "_id" : 3, "subject" : "History", "score" : 97 },
   { "_id" : 4, "subject" : "History", "score" : 71 },
   { "_id" : 5, "subject" : "History", "score" : 79 },
   { "_id" : 6, "subject" : "History", "score" : 83 }
] )
```

The following aggregation operation has two stages:

1. The `$match` stage excludes documents that have a `score` value of less than or equal to `80` to pass along the documents with `score` greater than `80` to the next stage.
2. The `$count` stage returns a count of the remaining documents in the aggregation pipeline and assigns the value to a field called `passing_scores`.

```
db.scores.aggregate( [
   { $match: { score: { $gt: 80 } } },
   { $count: "passing_scores" }
] )
```

The operation returns this result:

```
{ "passing_scores" : 4 }
```

If the input dataset is empty, `$count` doesn't return a result. The following example doesn't return a result because there are no documents with scores greater than `99`:

```
db.scores.aggregate( [
   { $match: { score: { $gt: 99 } } },
   { $count: "high_scores" }
] )
```

The C# examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see Get Started in the MongoDB .NET/C# Driver documentation.

The following `Movie` class models the documents in the `sample_mflix.movies` collection:

```
public class Movie
{
    public ObjectId Id { get; set; }

    public int Runtime { get; set; }

    public string Title { get; set; }

    public string Rated { get; set; }

    public List<string> Genres { get; set; }

    public string Plot { get; set; }

    public ImdbData Imdb { get; set; }

    public int Year { get; set; }

    public int Index { get; set; }

    public string[] Comments { get; set; }

    [BsonElement("lastupdated")]
    public DateTime LastUpdated { get; set; }
}
```

The C# classes on this page use Pascal case for their property names, but the field names in the MongoDB collection use camel case. To account for this difference, you can use the following code to register a `ConventionPack` when your application starts:

```
var camelCaseConvention = new ConventionPack { new CamelCaseElementNameConventio
ConventionRegistry.Register("CamelCase", camelCaseConvention, type => true);
```

`$count`

Count()

counts the number of input documents and returns a document with the count as its value:

To use the MongoDB .NET/C# driver to add a `$count` stage to an aggregation pipeline, call the Count() method on a `PipelineDefinition` object.

The following example creates a pipeline stage that counts the number of input documents and returns a document with the count as its value:

```
var pipeline = new EmptyPipelineDefinition<Movie>()
    .Count();
```

The Node.js examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see Get Started in the MongoDB Node.js driver documentation.

`$count`

counts the number of input documents from the `sample_mflix.movies` collection and returns a document containing the count

To use the MongoDB Node.js driver to add a `$count` stage to an aggregation pipeline, use the `$count` operator in a pipeline object.

The following example creates a pipeline stage that counts the number of input documents from the `sample_mflix.movies` collection and returns a document containing the count. The example then runs the aggregation pipeline:

```
const pipeline = [{ $count: "movies" }];

const cursor = collection.aggregate(pipeline);
return cursor;
```

Learn More

- `db.collection.countDocuments()`
- `$collStats`
- `db.collection.estimatedDocumentCount()`
- `count`
- `db.collection.count()`